

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



LỚP: CS112.Q11.KHTN

MÔN HỌC: PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

Luyện Tập Thiết Kế Thuật Toán
Nhóm 15

Nhóm 4:
Lê Văn Thức
Bảo Quý Định Tân

Giảng viên:
Nguyễn Thanh Sơn

1 Phân tích thuật toán

Đây là một bài toán thuộc thể loại **Lý thuyết trò chơi (Game Theory)**, cụ thể là một biến thể của trò chơi **Nim**. Ta nhận xét, bài toán này có các đặc điểm là **Impartial Games (Trò chơi công bằng)**: Trò chơi mà nước đi hợp lệ chỉ phụ thuộc vào trạng thái hiện tại của trò chơi, không phụ thuộc vào người đang chơi (ví dụ: cờ vua không phải là impartial game, nhưng bốc sỏi là có). Nên ta có thể sử dụng:

- **Định lý Sprague-Grundy** để giải quyết các bài toán Nim. Định lý phát biểu rằng mọi trạng thái của một trò chơi impartial đều tương đương với một đống sỏi Nim có kích thước g (gọi là giá trị Grundy hoặc nim-value).
- **Hàm MEX (Minimum Excluded value)**: Giá trị Grundy của một trạng thái u được tính bằng: $G(u) = \text{mex}(\{G(v) \mid u \rightarrow v \text{ là một nước đi hợp lệ}\})$.
- **Tổng XOR (Nim-sum)**: Giá trị Grundy của một trò chơi gồm nhiều trò chơi con độc lập (nhiều cột sỏi) bằng tổng XOR của các giá trị Grundy của từng trò chơi con: $G_{\text{total}} = G(s_1) \oplus G(s_2) \oplus \dots \oplus G(s_n)$.
 - Nếu $G_{\text{total}} > 0$: Người đi trước (Đạt) thắng.
 - Nếu $G_{\text{total}} = 0$: Người đi sau (Phú) thắng.

2 Subtask 1: $s_i \leq 10^5$

Ở subtask này, giá trị của các cột sỏi nhỏ ($N, s_i \leq 10^5$). Chúng ta có thể sử dụng phương pháp **Quy hoạch động (Dynamic Programming)** truyền thống để tính trước toàn bộ giá trị Grundy từ 0 đến 10^5 .

2.1 Phân tích thuật toán

- **Trạng thái**: Gọi $dp[i]$ là giá trị Grundy của cột sỏi có kích thước i .
- **Chuyển trạng thái**: Với mỗi i , ta xét các nước đi hợp lệ. Giới hạn bốc tối đa là $L = \lfloor \log_2(i) \rfloor + 1$.
- Các trạng thái tiếp theo có thể đến là: $i - 1, i - 2, \dots, i - L$.
- Công thức truy hồi:

$$dp[i] = \text{mex}(\{dp[i - 1], dp[i - 2], \dots, dp[i - L]\})$$

- Chúng ta sẽ tính lần lượt từ $i = 1$ đến $100,000$.

2.2 Độ phức tạp

Gọi $S_{\text{max}} = 10^5$ là giá trị lớn nhất của một cột sỏi.

2.2.1 Độ phức tạp Thời gian:

- Để tính $dp[i]$, ta cần duyệt qua các nước đi có thể. Số lượng nước đi tối đa là $\lfloor \log_2(i) \rfloor + 1$.
- Với i chạy từ 1 đến S_{max} , tổng số phép tính là:

$$\sum_{i=1}^{S_{max}} (\lfloor \log_2 i \rfloor + 1) \approx S_{max} \cdot \log_2(S_{max})$$

- Thay số: $10^5 \cdot 17 \approx 1.7 \cdot 10^6$ phép tính. Con số này rất nhỏ.
- Sau khi tiền xử lý (precompute), mỗi truy vấn cho một cột sỏi chỉ mất $O(1)$.
- Tổng thời gian: $O(S_{max} \cdot \log(S_{max}) + N)$.

2.2.2 Độ phức tạp Không gian:

- Cần một mảng kích thước S_{max} để lưu giá trị Grundy.
- Không gian: $O(S_{max})$ ($\approx 100,000$ phần tử integer).

2.3 Code Python (Subtask 1)

```
1 import sys
2
3 def solve_subtask_1():
4     MAX_S = 100005
5     grundy = [0] * MAX_S
6
7     for i in range(1, MAX_S):
8         limit = i.bit_length()
9
10        seen_grundy = set()
11
12        for x in range(1, limit + 1):
13            if i - x >= 0:
14                seen_grundy.add(grundy[i - x])
15
16        g = 0
17        while g in seen_grundy:
18            g += 1
19        grundy[i] = g
20
21    input_data = sys.stdin.read().split()
22    if not input_data: return
23
24    N = int(input_data[0])
25    s = [int(x) for x in input_data[1:]]
26
27    xor_sum = 0
28    for val in s:
29        xor_sum ^= grundy[val]
30
31    if xor_sum > 0:
```

```

32     print("Dat")
33     else:
34         print("Phu")
35
36 if __name__ == '__main__':
37     solve_subtask_1()

```

3 Subtask 2: $s_i \leq 10^9$

Ở subtask này, s_i quá lớn nên ta không thể tạo mảng ‘grundy’ kích thước 10^9 (sẽ bị lỗi bộ nhớ - Memory Limit Exceeded) và cũng không thể lặp 10^9 lần (sẽ bị lỗi thời gian - Time Limit Exceeded). Ta cần dùng giải thuật đệ quy dựa trên quy luật toán học (như đã phân tích ở câu trả lời trước).

Luật chơi: Từ n sỏi, được bốc x viên sao cho $1 \leq x \leq \lfloor \log_2(n) \rfloor + 1$. Đặt $L(n) = \lfloor \log_2(n) \rfloor + 1$.

Quan sát các giá trị Grundy đầu tiên:

- $G(0) = 0$
- $n = 1 : L(1) = 1$. Bốc 1. $G(1) = \text{mex}(G(0)) = 1$.
- $n = 2 : L(2) = 2$. Bốc 1, 2. $G(2) = \text{mex}(G(1), G(0)) = 2$.
- $n = 3 : L(3) = 2$. Bốc 1, 2. $G(3) = \text{mex}(G(2), G(1)) = 0$.
- $n = 4 : L(4) = 3$. Bốc 1..3. $G(4) = \text{mex}(G(3), G(2), G(1)) = 3$.
- $n = 5 : L(5) = 3$. Bốc 1..3. $G(5) = \text{mex}(G(4), G(3), G(2)) = 1$.
- $n = 6 : L(6) = 3$. Bốc 1..3. $G(6) = \text{mex}(G(5), G(4), G(3)) = 2$.
- $n = 7 : L(7) = 3$. Bốc 1..3. $G(7) = \text{mex}(G(6), G(5), G(4)) = 0$.
- $n = 8 : L(8) = 4$. Bốc 1..4. $G(8) = \text{mex}(0, 2, 1, 3) = 4$.

Ta nhận xét được quy luật: Xét một khoảng $[2^k, 2^{k+1} - 1]$. Trong khoảng này, giá trị giới hạn bốc tối đa là $M = k + 1$. Chu kỳ lặp lại của giá trị Grundy trong khoảng này sẽ là $P = M + 1 = k + 2$.

Từ đó suy ra công thức truy hồi để tính nhanh $G(n)$:

- Tìm k sao cho $2^k \leq n < 2^{k+1}$. Ta có $k = \lfloor \log_2 n \rfloor$.
- Độ dài chu kỳ $P = k + 2$.
- Tính vị trí tương đối trong chu kỳ: $r = (n - 2^k) \pmod{P}$.
- Nếu $r == 0$: $G(n) = k + 1$. Ta có thể chứng minh: tại vị trí này, sẽ được bốc $k + 1$, mà trong $k + 1$ số ta bốc, sẽ nằm ở khoảng trước. Mà ta cũng dễ dàng thấy rằng $k + 1$ số này có giá trị khác nhau và bắt đầu từ 0, nên sẽ cho ra giá trị MEX là $k + 1$.

- Nếu $r > 0$: Giá trị lặp lại từ một trạng thái trước đó nhỏ hơn 2^k . Cụ thể:

$$G(n) = G(2^k - P + r)$$

. Ta có thể chứng minh công thức này bằng cách: những số nằm trong chu kỳ đầu tiên của dãy $[2^k, 2^{k+1} - 1]$ mới này sẽ bốc $k + 1$ số, bao gồm số ở vị trí 2^k tức $G(2^k) = k + 1$ nên chắc chắn MEX của số hiện tại phải $< k + 1$. Mà biết sẽ bỏ đi một số rồi, chỉ còn bốc k số còn lại, thì ta thấy giống như nó tại tiếp tục nốt chu kỳ chưa được hoàn thành của phần cuối của dãy cũ, cho tới khi nào nó không bốc tới được vị trí 2^k .

3.1 Độ phức tạp

Gọi $S_{max} = 10^9$.

3.1.1 Độ phức tạp Thời gian:

- Mỗi lần gọi đệ quy, giá trị n giảm từ 2^k xuống dưới mức 2^k . Cụ thể, n giảm ít nhất một nửa (về độ lớn bit).
- Chiều sâu của cây đệ quy tối đa là số bit của S_{max} , tức là $\approx \log_2(10^9) \approx 30$.
- Với N cột sỏi, tổng thời gian là: $O(N \cdot \log(S_{max}))$.
- Thay số: $10^5 \times 30 = 3 \cdot 10^6$ phép tính.

3.1.2 Độ phức tạp Không gian:

- Do sử dụng đệ quy, ta tốn bộ nhớ Stack. Độ sâu tối đa là 30 tầng.
- Nếu dùng Memoization (lưu 'dict'), số lượng trạng thái lưu trữ tối đa cũng chỉ tỉ lệ với số tầng đệ quy nhân với N (trường hợp xấu nhất), nhưng thực tế ít hơn nhiều do các nhánh trùng nhau.
- Không gian: $O(N \cdot \log(S_{max}))$ (cho memoization) hoặc $O(\log(S_{max}))$ (chỉ tính stack đệ quy).

3.2 Code Python (Subtask 2 - Full Solution)

```

1 import sys
2
3 sys.setrecursionlimit(5000)
4
5 memo = {}
6
7 def get_grundy(n):
8     if n == 0:
9         return 0
10    if n in memo:
11        return memo[n]
12
13    k = n.bit_length() - 1
14
```

```

15     power_of_2 = 1 << k
16
17     P = k + 2
18
19     remainder = (n - power_of_2) % P
20
21     if remainder == 0:
22         res = k + 1
23     else:
24         prev_n = (power_of_2 + remainder) - P
25         res = get_grundy(prev_n)
26
27     memo[n] = res
28     return res
29
30 def solve():
31     input_data = sys.stdin.read().split()
32
33     if not input_data:
34         return
35
36     N = int(input_data[0])
37
38     xor_sum = 0
39
40     for i in range(1, len(input_data)):
41         s_i = int(input_data[i])
42         xor_sum ^= get_grundy(s_i)
43
44     if xor_sum > 0:
45         print("Dat")
46     else:
47         print("Phu")
48
49 if __name__ == '__main__':
50     solve()

```